Cognitive Computation (2022) 14:749–763
https://doi.org/10.1007/s12559-021-09987-7

Embeddings Evaluation Using a Novel Measure of Semantic Similarity

Anna Giabelli[1,2] · Lorenzo Malandri[2,3] · Fabio Mercorio[2,3] · Mario Mezzanzanica[2,3] · Navid Nobani[1,4]

Received: 19 July 2021 / Accepted: 17 December 2021 / Published online: 8 January 2022

Abstract
Lexical taxonomies and distributional representations are largely used to support a wide range of NLP applications, including semantic similarity measurements. Recently, several scholars have proposed new approaches to combine those resources into unified representation preserving distributional and knowledge-based lexical features. In this paper, we propose and implement `TaxoVec`, a novel approach to selecting word embeddings based on their ability to preserve taxonomic similarity. In `TaxoVec`, we first compute the pairwise semantic similarity between taxonomic words through a new measure we previously developed, the Hierarchical Semantic Similarity (`HSS`), which we show outperforms previous measures on several benchmark tasks. Then, we train several embedding models on a text corpus and select the best model, that is, the model that maximizes the correlation between the `HSS` and the cosine similarity of the pair of words that are in both the taxonomy and the corpus. To evaluate `TaxoVec`, we repeat the embedding selection process using three other semantic similarity benchmark measures. We use the vectors of the four selected embeddings as machine learning model features to perform several NLP tasks. The performances of those tasks constitute an extrinsic evaluation of the criteria for the selection of the best embedding (i.e. the adopted semantic similarity measure). Experimental results show that (i) `HSS` outperforms state-of-the-art measures for measuring semantic similarity in taxonomy on a benchmark intrinsic evaluation and (ii) the embedding selected through `TaxoVec` achieves a clear victory against embeddings selected by the competing measures on benchmark NLP tasks. We implemented the `HSS`, together with other benchmark measures of semantic similarity, as a full-fledged Python package called `TaxoSS`, whose documentation is available at https://pypi.org/project/TaxoSS.

Introduction and Motivation

In recent years, lexical taxonomies and distributional semantics have gained momentum in NLP applications. Lexical taxonomies are a natural method for organising human knowledge in a hierarchical form and provide a formal description of concepts and their relations, supporting syntactic and semantic exchanges. Contextually, word embeddings have gained remarkable popularity in computational linguistics. They are real-valued word representations that can extract linguistic patterns and lexical semantics from large corpora. However, despite their wide usage, finding a unified measure accounting for both knowledge-based and distributional resources is still an open problem.

Moreover, despite the extensive usage of word embeddings in a wide variety of modern Natural Language Processing [1], Understanding [2] and Generation [3] tasks, evaluating the performance of such representations is still an open debate. There is not a unique definition of what either an "effective" or a "performant" assessment measure is (see, e.g. [4, 5]); researchers tend to agree that the optimal vector representation depends on the use it will serve [6]. One thing is clear: given the high sensitivity of word embeddings to small changes in hyperparameters during their generation [7, 8], it is crucial to have a reliable evaluation measure that can produce a performant semantic representation based on the intended scope and the information they have to embed.

In our case, selecting a word vector model that represents and preserves taxonomic similarity relations will allow us to generate a unified representation of knowledge-based and data-driven lexical features and, simultaneously, enable several NLP applications. Some of them are related to the

✉ Lorenzo Malandri
lorenzo.malandri@unimib.it

1 Dept. of Informatics, Systems & Communication, Univ. of Milan-Bicocca, Milan, Italy

2 CRISP Research Centre, University of Milano Bicocca, Milan, Italy

3 Dept. of Statistics and Quantitative Methods, Univ. of Milan-Bicocca, Milan, Italy

4 Digital Attitude, Milan, Italy

maintenance and taxonomy update, such as taxonomy refinement, enrichment, and learning. Others are downstream tasks that rely on underlying structured knowledge representation, such as personalised recommendations for online retailers [9], query understanding for search engines [10], and text understanding [11], to cite a few of them. For this reason, we developed `TaxoVec`, an embedding selection framework driven by a semantic similarity measure between taxonomic elements: the Hierarchical Semantic Similarity.

Semantic similarity represents a particular case of semantic relatedness [12], considering only the co-hyponymy and synonymy relations. For instance, the words *cat* and *tiger* are more similar than the words *jungle* and *tiger*, while the latter pair seems to be more related. The semantic similarity strongly depends on the context. For this reason, it is useful to find an automated method to compute the similarity between words in a domain-dependent taxonomy. In "Preliminaries and State-of-the-Art", we present different approaches for measuring semantic similarity in a taxonomy. Despite all of them being relevant, they have two main drawbacks: first, when a word has multiple senses, those methods compute a value of similarity for each word sense and then consider only the highest; second, while they consider the structure of the taxonomy, thus the relationship between taxonomic concepts, none of those measures accounts for the number of words belonging to those concepts. In [13], we developed the `HSS`, a measure that accounts for those two limitations. The `HSS` has proven to be useful in several applications, such as taxonomy enrichment [14–16], refinement [17], and job-skill mismatch analysis in the labour market field [18, 19]. In this paper, we evaluate the (i) `HSS` as a measure of semantic similarity and (ii) `TaxoVec` as a method for selecting taxonomy-aware word embeddings. We perform an extensive set of intrinsic and extrinsic evaluation tasks on well-known benchmarks to evaluate the `HSS` and `TaxoVec` against the state-of-the-art measures of taxonomic semantic similarity and methods for embedding evaluation. Moreover, we develop a Python package[1], fully deployed and available to the whole community, which allows us to easily compute the `HSS` (and all the other semantic similarity metrics considered) between any pair of WordNet nouns.

Motivation

Recently, we have witnessed a clear dominance of learning-based methods in NLP applications [20]. Although subsymbolic AI (i.e. machine learning) has been shown to outperform symbolic AI (i.e. knowledge-based AI), in retrieving linguistic patterns from a large quantity of data and in making predictions, those methods still suffer from dependency issues, i.e. they require large amounts of training data and are domain dependent [21, 22]. This issue is particularly severe in domains where it is difficult to retrieve labelled data, and there is extensive use of jargon and linguistic features that subsymbolic AI struggles to learn, such as rhetoric, irrealis moods, and metaphors [23, 24]. For those reasons, enhancing distributional models with lexical knowledge creates noteworthy benefits to their use as input features in several downstream machine learning tasks[25].

However, lexical taxonomies are manually built and, as a consequence, their creation and update are time-consuming, error-prone, require domain-specific knowledge and usually have low coverage [26]. The use of taxonomy-aware embeddings could be beneficial to automatically infer semantic information from domain-specific text corpora to build, update or maintain lexical taxonomies [13, 14].

Contribution

The contributions of this work proceed in three directions.

- First, we evaluate a new measure of semantic similarity in taxonomies, namely `HSS`, that we developed in [13]. To this aim, we employ classical benchmarks, showing that the `HSS` outperforms the current state-of-the-art semantic similarity measures in taxonomies;
- Second, the `HSS`, together with all the other benchmark metrics described in this paper, is implemented as a full-fledged Python package, available to the whole community, for easily computing the pairwise semantic similarity between WordNet nouns.
- Third, we use the `HSS` as a driver for the choice of a taxonomy-aware vector model, i.e. we generate several embeddings, and we select the one that best preserves the taxonomic similarity between words. We show that this vector model outperforms embeddings selected through other state-of-the-art embedding evaluation criteria on several downstream NLP tasks, performed using the word vectors as input features.

The remainder of the paper is organised as follows. In "Preliminaries and State-of-the-Art", we present the related work and a formal definition of taxonomy, which is useful for describing `TaxoVec`. In "Methodology", we present the three main methodological contributions of this paper: the `HSS`, the Python package, and the embeddings evaluation method. In "Experimental Results", we present and discuss the experimental results, and "Conclusion and Future Work" concludes the paper and presents further research steps.

[1] https://pypi.org/project/TaxoSS

Preliminaries and State-of-the-Art

In this section, we survey the primary studies related to taxonomy-based semantic similarity and word embeddings evaluation, showing how these techniques are related to HSS. Then, some background notions are introduced to better explain how TaxoVec works.

A Definition of Taxonomy

To better describe TaxoVec, in the following section, we introduce and formalise the notion of semantic hierarchy. Based on [27], the building blocks of a semantic hierarchy for a specific domain S can be defined as follows:

- A set $\mathcal{C}$ of concepts (nodes) c belonging to domain D;
- A set W of possible words (or entities) belonging to the domain D. Each word w can be assigned to none, one or multiple concepts $c \in \mathcal{C}$;
- A taxonomic relation (edge) $H^c \subseteq \mathcal{C} \times \mathcal{C}$, which is a directed relation between elements in $\mathcal{C}$, i.e. where $H^c(c_1, c_2)$ indicates that c_1 is a subconcept of c_2. The relation $H^c(c_1, c_2)$ is also called $IS-A$ relation (c_1 $IS-A$ subconcept of c_2).

The direction of H^c is from the most specific concept towards the most generic. As a consequence, the concepts with the finest granularity have an in-degree of 0, i.e. they do not have any incoming edges.

Taxonomy-Based Semantic Similarity

In the past literature, several researchers attempted to measure semantic similarity between words in a taxonomy, expressed as a similarity between the concepts to which the two words belong. Those measures can be roughly divided into two main categories: those who exploit the path connecting two concepts and those who are based on the information content (IC) of the concepts. The most important measures belonging to these two categories, as assessed by different researchers [28, 29], are as follows.

Path-Based Measures These measures employ the path connecting two concepts to estimate their similarity.

The simplest approach is to use the *shortest path* to assign a similarity score:

$$sim_{sp}(c_1, c_2) = \frac{1}{\phi(c_1, c_2) + 1} \tag{1}$$

where $\phi(c_1, c_2)$ is the shortest path between c_1 and c_2.

Leacock and Chodorow [30] scale the path similarity by the depth of the taxonomy:

$$sim_{lc}(c_1, c_2) = -\log \frac{\phi(c_1, c_2)}{2 \times max\_depth} \tag{2}$$

Wu and Palmer [31] consider the position of $LCA(c_1, c_2)$, the Lowest Common Ancestor of c_1 and c_2:

$$sim_{wup}(c_1, c_2) = \frac{2 \times \phi(r, LCA)}{\phi(c_1, LCA) + \phi(c_2, LCA) + 2 \times \phi(r, LCA)} \tag{3}$$

where r is the root node. Note that to simplify the notation, we refer to $LCA(c_1, c_2)$ simply as LCA. For this class of methods in the case of polysemy, i.e. words belonging to several concepts, the minimum $\phi(c_1, c_2)$ is considered, thus the maximum similarity.

Information content-based measures The information content-based approach was introduced by Resnik [12]. According to information theory, the information content—IC (or self-information)—of concept $c \in \mathcal{C}$ can be approximated by its negative log-likelihood:

$$IC(c) = -\log p(c) \tag{4}$$

where $p(c)$ is the probability of encountering the concept c.

In the case of a taxonomy, $p(c)$ is monotonic and increases with the taxonomy rank: if c_1 $IS-A$ c_2, then $p(c_1) \leq p(c_2)$. Moreover, the probability of the root node is 1. Concept probabilities are computed simply as relative frequencies in a text corpus:

$$\hat{p}(c) = \frac{freq(c)}{N} = \frac{\Sigma_{n \in words(c)}}{count(n)} N \tag{5}$$

where $words(c)$ is the set of words subsumed by concept c and N is the total number of occurrences of words in the corpus that are also present in the taxonomy. Therefore, *Resnik* defines the similarity between two concepts c_1 and c_2 as:

$$sim_{res}(c_1, c_2) = IC(LCA) \tag{6}$$

and the similarity between two words as:

$$sim_{res}(w_1, w_2) = \max_{\substack{c_1 \in s(w_1), \\ c_2 \in s(w_2)}} IC(LCA) \tag{7}$$

where $s(w_1)$ and $s(w_2)$ are all the possible senses of w_1 and w_2, respectively.

Jiang-Conrath [32] built a measure of similarity using the self-information of c_1 and c_2 as well:

$$sim_{jcn}(c_1, c_2) = \frac{1}{IC_{res}(c_1) + IC_{res}(c_2) - 2 \times IC_{res}(LCA)} \quad (8)$$

and a similar measure is built by Lin [33]:

$$sim_{lin}(c_1, c_2) = \frac{2 \times IC_{res}(LCA)}{IC_{res}(c_1) + IC_{res}(c_2)} \quad (9)$$

Similar to Resnik, these last two methods consider the two concepts with the highest Resnik similarity in the case of multiple word senses.

Other information content-based measures, sometimes called *intrinsic information content-measures*, use the structure of the taxonomy, instead of an external corpus frequency, to compute $\hat{p}(c)$. For example, Seco et al. [34] compute the information content IC(c) as:

$$IC_{seco}(c) = 1 - \frac{\log(|descendants(c)| + 1)}{\log N_c} \quad (10)$$

where N_c is the number of concepts in the taxonomy and $|descendants(c)|$ the number of subconcepts of c.

They have two main drawbacks: first, when a word has multiple senses, those methods compute a value of similarity for each word sense and then consider only the highest; second, while they consider the structure of the taxonomy, i.e. the relationship between taxonomic concepts, none of those measures account for the number of words belonging to those concepts.

In "Step 1: Hierarchical Semantic Similarity (HSS)", we present the `HSS`, a measure for semantic similarity considering both multiple word senses and the cardinality of taxonomic concepts in terms of entities (words) that they subsume. This measure, which we developed in [13], has proven to be useful in several applications, such as taxonomy enrichment [14, 15], refinement [17], and job-skill mismatch analysis in the field of labour market [18].

Word Embeddings

Word embeddings are vector representations of words based on the hypothesis that words occurring in a similar context tend to have a similar meaning. Words are represented by semantic vectors, which are usually derived from a large corpus using co-occurrence statistics, and their use improves learning algorithms in many NLP tasks. Two powerful methods to induce word embeddings are neural network training [35, 36] and co-occurrence matrix factorisation [37, 38].

These techniques consider each word as a distinct vector and ignore the morphological similarity among them. More recently, in [39] the authors developed a version of the continuous *skipgram* model [36] that considers subword information. This architecture, called fastText, is an extension of word2vec for scalable word representation and classification. One of its major improvements to word2vec is to consider subword information by representing each word as the sum of its character *n*-gram vectors. Formally, given a word w, and a dictionary of size G, G_w is the set of n-grams of size G appearing in w. Denoted as z_g the vector representation of the n-gram g, w will be represented as the sum of the vector representation of its n-grams and the score associated with the word w as:

$$\int(\mathrm{w}, \mathrm{c}) = \sum_{g \in G_w} z_g^{\mathrm{T}} v_c \quad (11)$$

where v_c is the vector representing the context. This simple representation allows information between words to be shared, and this makes it useful for representing rare words, typos, and words with the same root.

Other embedding models have been evaluated along with fastText. Nevertheless, none of them fit our conditions. Neither classical embedding models [36, 38] nor embeddings specifically designed to fit taxonomic data [40, 41] consider subword information. Moreover, they cannot be easily bonded with external sources in their generation phase, and this would reduce the flexibility of `TaxoVec`. We discarded hyperbolic and spherical embeddings such as HyperVec [42] or JoSe [43], since (i) they also do not consider subword information, which is important working with short text and many words with the same root (e.g. engineer-engineering, developer-developing) as OJVs, and (ii) HyperVec uses hypernym–hyponym relationships for training, while we train our models on a text corpus which has not such relationships. Finally, we considered context embeddings (see, e.g. [44]). However, contextual embeddings represent words based on their context, thus capturing the uses of words across varied contexts. Thus is not suitable for our case, where we aim to compare words in a corpus and their similarity with taxonomy words, with a given sense.

Evaluation Methods for Word Embeddings

When it comes to classifying embedding evaluation methods, researchers almost univocally agree on *intrinsic* vs *extrinsic* division (see, e.g. [6, 45]).

Intrinsic Metrics attempt to increase the syntactic or semantic relationships between words, either by directly obtaining human judgments (comparative intrinsic evaluation [6]) or by comparing the aggregated results with the preconstructed datasets (absolute intrinsic evaluation). A well-known intrinsic evaluation method is Semantic Relatedness [6, 46], which evaluates the performance of a vector model by the correlation between the cosine similarity between pairs of word vectors and human scores of relatedness between them. A similar method is used in semantic similarity [47, 48], which reduces

relatedness to similarity. Some of the limitations of *intrinsic metrics* are (i) suffering from word sense ambiguity(faced by a human tester) and subjectivity (see, e.g. [45, 49]); (ii) facing difficulties in finding significant differences between models due to the small size and low inter-annotator agreement of existing datasets and; (iii) the need to construct judgement datasets for each language [50] and each domain, given that word meanings can change greatly across different domains. Moreover, updating and upscaling handcrafted resources, such the similarity human scores, is costly, time expensive, and error-prone.

Extrinsic metrics on the other hand, perform the evaluation by using embeddings as features for specific downstream tasks. For instance, question deduplication [51], Part-Of-Speech (POS) tagging, Language Modelling [52] and Named Entity Recognition (NER) [53], just to cite a few. As a drawback, extrinsic metrics are (i) computationally heavy [45], (ii) have high complexity of creating gold standard datasets for downstream tasks, and (iii) lack performance consistency on downstream tasks [6].

The evaluation method we propose in this paper is similar to *intrinsic thesaurus-based evaluation metrics*, as it uses an expert-constructed taxonomy to evaluate the word vectors as an intrinsic metric [46, 48]. However, typically, these kinds of approaches evaluate embeddings by their correlation with manually crafted lexical resources, such as expert ratings of similarity or relatedness between hierarchical elements. However, those resources are usually limited and hard to create and maintain. Furthermore, human similarity judgments ex novo evaluate the semantic relatedness between taxonomic elements, but the structure of the taxonomy already encodes information about the relations between its elements. In this research, instead, we exploit the information encoded in an existing taxonomy to build a benchmark for word embeddings evaluation.

To the best of our knowledge, this is the first attempt to encode similarity relationships from a manually built semantic hierarchy into a distributed word representation automatically extracted from a text corpus.

Methodology

In this section, we present `TaxoVec`, which is composed of three steps. In *Step 1*, we define a measure of similarity within a semantic hierarchy, which serves as a basis for the embeddings evaluation. In *Step 2*, we create a tool for computing semantic similarity using our metric, the `HSS`, and the other state-of-the-art measures. In *Step 3*, we generate embeddings from a large text corpus, and we identify which one better preserves the taxonomic relations of semantic similarity; then, we utilise the embeddings to perform four NLP tasks: *categorisation*, *sentiment classification*, *hypernym detection*, and *synonym detection*.

Step 1: Hierarchical Semantic Similarity (`HSS`)

In this section, we present and evaluate the measure of semantic similarity in taxonomies that we introduced in [13], the `HSS`, which is used for the evaluation of the embeddings in "Evaluation of the Best Embedding". Similar to [34], we compute $\hat{p}(c)$ using an intrinsic measure, exploiting the structure of the taxonomy instead of an external corpus. However, the `HSS`, unlike [34], which used only the number of taxonomic concepts, also considers also the entities of the taxonomy:

$$\hat{p}(c) = \frac{N_c}{N} \tag{12}$$

where N is the cardinality, i.e. the number of entities (words) of the taxonomy and N_c is the sum of the cardinality of the concept c with the cardinality of all its hyponyms. Note that $\hat{p}(c)$ is monotonic and increases with granularity, thus respecting our definition of p (see "Taxonomy-Based Semantic Similarity").

Now, given two words w_1 and w_2, Resnik defines $c_1 \in s(w_1)$ and $c_2 \in s(w_2)$ to be all the concepts containing w_1 and w_2, respectively, i.e. the *senses* of w_1 and w_2. Therefore, there are $|s(w_1)| \times |s(w_2)|$ possible combinations of their word senses, where $|s(w_1)|$ and $|s(w_2)|$ are the cardinality of $s(w_1)$ and $s(w_2)$, respectively. We can now define $\mathcal{L}$ as the set of all the lowest common ancestors for all the combinations of $c_1 \in s(w_1)$ and $c_2 \in s(w_2)$.

The `HSS` between the words w_1 and w_2 can, therefore, be defined as:

$$sim_{\texttt{HSS}}(w_1, w_2) = \sum_{\ell \in \mathcal{L}} \hat{p}\big(\ell = LCA \mid w_1, w_2\big) \times IC(LCA) \tag{13}$$

where $\hat{p}\big(\ell = LCA \mid w_1, w_2\big)$ is the probability of LCA being the lowest common ancestor of w_1, w_2, and can be computed as follows applying the Bayes theorem:

$$\hat{p}\big(\ell = LCA \mid w_1, w_2\big) = \frac{\hat{p}\big(w_1, w_2 \mid \ell = LCA\big)\hat{p}(LCA)}{\hat{p}\big(w_1, w_2\big)} \tag{14}$$

We define N_ℓ as the cardinality of ℓ and all its descendants. Now, we can rewrite the numerator of Eq. 14 as:

$$\hat{p}\big(w_1, w_2 \mid \ell = LCA\big)\hat{p}(\ell = LCA) = \frac{S_{<w_1, w_2> \in \ell}}{|descendants(\ell)|^2} \times \frac{N_\ell}{N} \tag{15}$$

where the first leg of the *rhs* is the class conditional probability of the pair $< w_1, w_2 >$ having ℓ as the lowest common ancestor and the second leg is the marginal probability of the class ℓ. The term $|descendants(\ell)|$ represents the

number of subconcepts of ℓ. Since we could have at most one word sense w_i for each concept c, $|descendants(\ell)|^2$ represents the maximum number of combinations of word senses $< w_1, w_2 >$ which have ℓ as lowest common ancestor. $S_{<w_1,w_2>\in LCA}$ is the number of pairs of senses of words w_1 and w_2 which have LCA as the lower common ancestor.

The denominator of Eq. 14 can be written accordingly as:

$$\hat{p}(w_1, w_2) = \sum_{k \in \mathcal{L}} \frac{S_{<w_1,w_2>\in k}}{|descendants(k)|^2} \tag{16}$$

Evaluation of the `HSS`

This step aims to compare the `HSS` with the previous measures of semantic similarity presented in "Preliminaries and State-of-the-Art": WUP, LC, shortest path, Resnik, Jiang-Conrath, Lin, and Seco. The evaluation is composed of two tasks: *semantic similarity* and *word clustering* (also called *concept categorisation*). In the first one, we measure how the different measures of semantic similarity are correlated with a gold benchmark. In the second one, we measure how words belonging to the same semantic group in a gold benchmark are similar to each other. To do this, following the state-of-the-art [28, 54], we adopt a set of human-annotated resources, considered the gold benchmark for semantic similarity and word categorisation.

Step 2: A Tool for Computing Semantic Similarity

Semantic similarity can be useful for a wide number of tasks. The embedding selection that we perform in "Step 3: Embeddings Evaluation and Selection of the Best Embedding" is only one of them. However, as highlighted in the introduction, handcrafted semantic similarity resources are usually not updated and are limited in coverage. For this reason, we decided to implement the `HSS` and all the other automatic semantic similarity measures considered in "Taxonomy-Based Semantic Similarity" as a full-fledged Python package. This is going to facilitate the user that wants to use it. This library, called *TaxoSS* (Taxonomic Semantic Similarity), allows computing taxonomic-based similarity (using WordNet 3.0) as well as corpus-based similarity measures. For the latter, there is a default measure based on the English Wikipedia dump of 2008, but the user can also use a different corpus. All the implementation details are reported in " Step 2: A Tool for Computing Semantic Similarity".

Step 3: Embeddings Evaluation and Selection of the Best Embedding

In this step, we generate a variety of vector representations of a large text corpus through FastText and, by an intrinsic evaluation, we select the one that better represents the taxonomy, that is, we want the similarity between word vectors to reflect as much as possible the semantic similarity between words in the taxonomy. Then, we perform an extrinsic evaluation to compare the embedding model selected through `HSS` with the models selected by other semantic similarity measures.

Embeddings Selection

Following the previous literature [6, 46, 48], which correlates the cosine similarity between pairs of word vectors with human scores of relatedness/similarity, we assess the goodness of a vector model by the Pearson correlation coefficient between the cosine similarity of pairs of word vectors and their taxonomic semantic similarity. The taxonomic semantic similarity can be measured with all the metrics presented in "Preliminaries and State-of-the-Art" or by experts, as in the case of human-crafted resources.

This kind of evaluation was developed in [46] with the name of *Semantic Relatedness*, where the authors use MEN as a measure of similarity. In [48], the authors present a new dataset, SimLex-999, which explicitly measures similarity rather than relatedness, to foster the development of models that reflect word similarity instead of relatedness.

In this step, we select the best embeddings that maximise the correlation between cosine similarity and semantic similarity as it is computed by `HSS`, MEN, and SimLex-999. In addition, given its upstanding performances in step 1, we add the similarity computed using WUP as an additional criterion for a total of four selected embedding models.

Given that with `HSS` and WUP, we can compute the similarity for each pair of terms in the taxonomy, for this evaluation we have $(n \times (n-1))/2$ pairs of terms for each vector model, where n is the number of words present both in the taxonomy and the text corpus. In our case, the number of common words is 53,451, for a total of 1,428,477,975 possible word pairs for each model, which would makes the computation intractable. To reduce the number of samples, we start from 0 pairs and, following [55], we increase the sample size until the Pearson correlation stabilises. The process of choosing the number of pairs is detailed in "Semantic Similarity" section.

Evaluation of the Best Embedding

To evaluate the selected embeddings, we rely on the hypothesis that the embedding that better preserves the semantic knowledge encoded in WordNet will improve the performance of several downstream NLP tasks [56]. Therefore, the performance of the four selected embeddings in the NLP tasks presented in "Step 3: Embeddings Evaluation and Selection of The Best Embedding" constitutes an extrinsic evaluation of the criteria used to select the vector model, i.e. the similarity measure employed.

Word embeddings can be used as feature vectors of supervised machine learning algorithms used in various NLP tasks. Relying on the hypothesis that unifying lexical and distributional resources in a taxonomy-aware vector model is beneficial for downstream NLP tasks [4, 46, 56]; in this section, we examine different queries related to well-known NLP applications.

Categorisation Categorisation is a natural task to verify to what extent the selected embedding preserves the taxonomic similarity. Since dealing with taxonomic data, this task is especially crucial since having a performant embedding means that the similarity values can reflect the hierarchical relationships among words and preserve the hypernym–hyponym links. Following the method described in [46], we cluster the vectors from each selected embedding by applying a clustering algorithm, and we measure the purity of each cluster. In "Categorisation", we describe the process and the metric used to perform this task.

Sentiment Classification Given the strong bond between lexicons and sentiment analysis, for instance, in word polarity disambiguation [57] or neutral and ambivalent sentences detection [58, 59], we carry out a sentiment classification task on three customer review datasets, with two binary classifications and one multiclass sentiment classification. "Sentiment Classification" describes in detail the datasets used in this task and their characteristics, together with comments on the achieved results.

Hypernym Detection In this task, we examine to what extent the chosen embeddings (see "Step 3: Embeddings Evaluation and Section of the Best Embedding") can be utilised to identify the hyponym/hypernym relation between two words in a given pair. In "Hypernym Detection", we describe the process for generating the training and test data while commenting on the results of this task on each dataset.

Synonym Detection This task aims to evaluate the impact of the input word embeddings on the performance of a classifier that is trained to detect the synonym pair of words. "Synonym Detection" provides details about how positive and negative pairs are generated and used to train the classifier.

Experimental Results

Our experiments rely on a lexical taxonomy and a corpus:

- **Taxonomy:** WordNet [60] provides a structured hierarchy of meanings (senses) and synsets (a collection of words belonging to a specific context). We used the implementation of WordNet inside the NLTK library [61] while calculating the required information using NLTK's native functions (e.g. calculating the lowest common ancestor) and custom functions (e.g. calculation of cardinality).
- **Corpus:** English Wikipedia dump of the year 2008. The main reason for not choosing a more recent dump is that the last release of WordNet 3.0 (the version we used for our experiments) was from 2006, making the use of a newer Wikipedia dump version unnecessary. The dump used already includes the preprocessed data version that we used without performing further cleaning.

Step 1: Evaluation of the `HSS`

In this section, we compare the `HSS` with the other semantic similarity measures presented in "Preliminaries and State-of-the-Art" through two tasks: *semantic similarity* and *word clustering* (also called *concept categorisation*) introduced in "Step 1: Hierarchical Semantic Similarity (HSS)".

Semantic Similarity

For this task, we consider Pearson and Spearman's correlation between the pairwise similarity assessed by the similarity metrics and by humans. The six well-known resources considered in this paper are MEN, MC30, WSS (the similarity portion of WordSim-353), SimLex-999, MT287, and MT771. Among them, MEN is proposed as a *semantic relatedness* dataset, even though it does not distinguish between similarity and relatedness, conflating the two [56]; for this reason, and given its relevance in the literature, we decided to use it for the evaluation together with semantic similarity datasets.

A brief description of the employed datasets is reported below:

- **MEN**: 3000 random pairs that appeared at least 700 times in ukWaC and Wackypedia corpora with human-assigned similarity judgements, obtained by crowdsourcing [62].
- **MC30**: 30 pairs selected from the RG65 dataset [63] with the similarity scores obtained from 38 participants [60].
- **WSS**: A part of the WordSim353 dataset provided by [64] that addresses only the similarity, unlike the original dataset that has both relatedness and similarity [47].
- **SimLex-999**: Containing 111 adjective pairs, 666 nouns pairs and 222 verb pairs [48].
- **MT287**: 287 pairs evaluated using human annotators from Mechanical Turk workers [65].
- **MT771**: 771 word pairs evaluated using human annotators from Mechanical Turk workers, with an average of 20 worker ratings for each word pair [66].

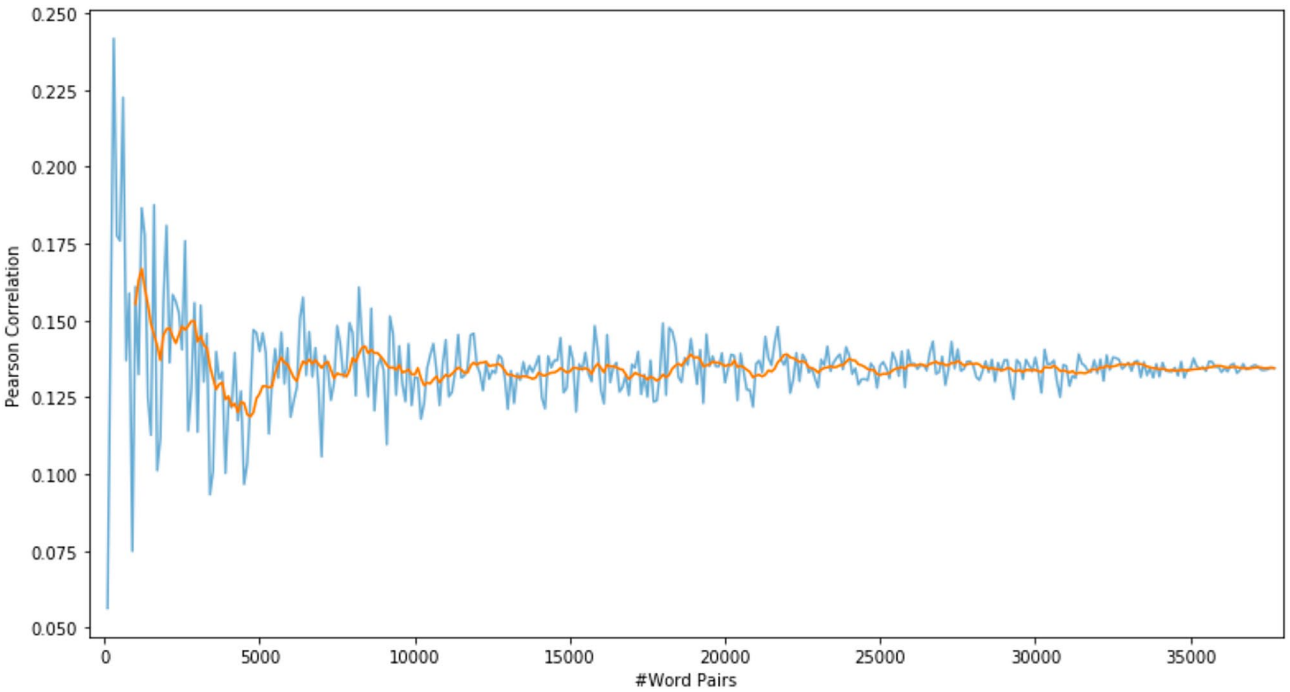

Fig. 1 Variation in Pearson Correlation of HSS score and cosine similarity Vs. the number of word pairs. The orange line indicates the rolling average of 100

Choosing the number of pairs To define the minimum number of pairs required for our experiments, after randomly generating 100k pairs presented both in WordNet and our corpus vocabulary, we recursively generate samples of pairs while increasing the number of samples by 100 in each step. Figure 1 shows the Pearson correlation of the HSS score and cosine similarity of each paired sample, while the orange line indicates the rolling average of 100. To define the Point of Stability (PoS) (i.e. a point from which the correlation remains within the POS), we considered a Corridor of Stability (CoS) of ± 0.01. Although the rule of thumb proposed by [67] and the work of [55] suggest a larger CoS (between 0.05 and 0.1) due to difficulties in achieving a tighter CoS mainly caused by the cost of the additional samples. Since such a cost is not relevant in our case, we decided to consider COS as ± 0.01, which results in a PoS of 35,000.

Comparative Results Table 1 shows the results of calculating Pearson and Spearman correlation coefficients among six datasets annotated by humans and eight similarity scores. HSS outperforms the other measures (except for the SimLex-999 dataset), both in terms of Pearson and Spearman correlations with the human-annotated datasets. The closest performance to HSS is achieved by the WUP metric. These results confirm the performance superiority of HSS to the SOTA measures (both path-based and information-content-based measures). To conduct this experiment, we use the implementation of WUP, LC, and shortest path in the NLTK library, while in the case of Resnik, Jiang-Conrath, Lin, and Seco (Resnik using Seco information content), we implemented them in Python. Table 1 also reports the average time for calculating the similarity between two words (in seconds), calculated as the average time in seconds for 1,000 random pairs. Our model has the best performance in terms of computational time since it is twice as fast as the second-best performer that is the Resnik measure, which uses information content calculated by Seco et al. [34].

Word Clustering

To measure the similarity between words belonging to the same concept, we compute the silhouette coefficient of the cluster of words belonging to the same category in human-annotated datasets. The three datasets used are as follows:

- **ESSLLI** (European Summer School in Logic, Language and Information): 45 words divided into 9 categories [68].
- **AP** (Almuhareb and Poesio): 402 words divided into 21 semantic classes [69].
- **BM** (Battig and Montague) 5,321 concepts belonging to 56 categories [70].

Table 1 Semantic similarity

	HSS (ours)		WUP [31]		LC [30]		**Shortest Path**	
	Pearson	Spearman	Pearson	Spearman	Pearson	Spearman	Pearson	Spearman
MEN [62]	**0.41**	**0.33**	0.36	**0.33**	0.14	0.05	0.07	0.03
MC30 [63]	**0.74**	0.69	**0.74**	**0.73**	0.33	0.21	0.22	0.3
WSS [47]	**0.68**	**0.65**	0.58	0.59	0.36	0.23	0.16	0.1
SimLex-999 [48]	0.4	0.38	**0.45**	**0.43**	0.26	0.15	0.2	0.16
MT287 [65]	**0.46**	**0.31**	0.4	0.28	0.26	0.12	0.11	0.11
MT771 [66]	**0.44**	0.4	0.43	**0.49**	0.06	0.02	0.1	0.13
Time per pair (s)	0.0007		0.008		0.0055		0.0064	
	Resnik [12]		Jiang-Conrath [32]		Lin [33]		Seco [34]	
	Pearson	Spearman	Pearson	Spearman	Pearson	Spearman	Pearson	Spearman
MEN [62]	0.05	0.03	-0.05	-0.04	0.05	0.04	-0.01	0.03
MC30 [63]	0.13	0.03	-0.06	-0.01	0.05	0.01	0.13	-0.09
WSS [47]	0.02	-0.03	0.04	0.06	0.03	0.06	-0.01	-0.04
SimLex-999 [48]	-0.04	-0.04	0.12	0.14	0.12	0.14	-0.02	-0.08
MT287 [65]	0.03	0.04	0.18	0.16	0.22	0.17	0	-0.06
MT771 [66]	0	-0.01	0	0	0	0	-0.05	-0.03
Time per pair (s)	0.5586		0.551		0.5866		0.0013	

The silhouette coefficient is a cluster validity measure that compares how similar an object is to its cluster with how similar it is to other clusters. The silhouette is computed as:

$$Silhouette(i) = \frac{b(i) - a(i)}{\max(b(i), a(i))} \tag{17}$$

where $a(i)$ is the mean distance between i and all other data points in the same cluster, and $b(i)$ is the smallest mean distance of i to all points in any other cluster. The silhouette ranges from -1 to $+1$: a value near to $+1$ indicates that the object is well matched to its own cluster and poorly matched to neighbouring clusters, and vice versa for a value near -1 [71].

Comparative Results Table 2 shows the word clustering task among the three datasets described above and eight similarity scores. HSS outperforms all the other measures, and only LC reaches an equivalent performance in terms of the silhouette on the BM dataset.

These results confirm the performance superiority of the HSS with respect to the SOTA measures.

Table 2 Cluster purity

	HSS (ours)	WUP [31]	LC [30]	Shortest Path
ESSLLI [68]	**0.18**	0.11	-0.01	-0.01
AP [69]	**0.33**	0.09	0.04	0.01
BM [70]	**0.09**	0.02	**0.09**	-0.03
	Resnik [12]	Jiang-Conrath [32]	Lin [33]	Seco [34]
ESSLLI [68]	-0.05	-0.37	-0.06	-0.29
AP [69]	-0.09	-0.13	-0.15	-0.32
BM [70]	-0.11	-0.39	-0.19	-0.42

Step 2: A Tool for Computing Semantic Similarity

The Python library *TaxoSS* that we created allows the user to easily compute semantic similarity between concepts using eight different measures: HSS, WUP, LC, Shortest Path, Resnik, Jiang-Conrath, Lin, and Seco.

Figure 2 shows the use of the package for computing the semantic similarity between two words using different metrics. Figure 2 also shows how the users can use their corpus to compute the similarity through corpus-based similarity measures.

Figure 2 reports also the table of the average times requested for each similarity measure to compute 100 similarities between 100 pairs of words. The times are similar for all the methods, ranging from 5.29 up to 5.98.

	HSS	WUP [31]	LC [30]	Shortest Path
100 pairs	5.77	5.29	5.38	5.29
	Resnik [12]	Jiang-Conrath [32]	Lin [33]	Seco [34]
100 pairs	5.80	5.88	5.98	5.55

```
from TaxoSS.functions import semantic_similarity
semantic_similarity('brother', 'sister', 'hss')

3.353513521371089
```

```
from TaxoSS.functions import semantic_similarity
semantic_similarity('cat', 'dog', 'resnik')

6.169410755220327

from TaxoSS.calculate_IC import calculate_IC
calculate_IC('data/corpus_test.csv', 'venv/lib/python3.6/site-packages/TaxoSS/data/test_IC.csv')
semantic_similarity('cat', 'dog', 'resnik', 'venv/lib/python3.6/site-packages/TaxoSS/data/test_IC.csv')

3.5077209766856137
```

Fig. 2 An example of the use of the *semantic similarity* function with the HSS metric and the Resnik metric with the use of an ad hoc Information-Content file created through a corpus of choice. In the table, it is shown the mean time (seconds) requested for computing the semantic similarity of 100 pairs with TaxoSS

	HSS	WUP [31]	LC [30]	Shortest Path
100 pairs	5.77	5.29	5.38	5.29
	Resnik [12]	Jiang-Conrath [32]	Lin [33]	Seco [34]
100 pairs	5.80	5.88	5.98	5.55

Step 3: Embeddings Evaluation and Selection of the Best Embedding

In this section, we generate 80 different embedding models with fastText. Among them, we select the four that better correlate their cosine similarity with the semantic similarity, measured with the HSS, MEN, SimLex-999, and WUP. To evaluate these four models and compare the semantic similarity measures used to select them, we use their vectors as input features in four downstream NLP tasks. The four NLP tasks are, in the order of presentation: categorisation, sentiment classification, hypernym detection, and synonym detection.

Generation of Embeddings We trained our vector models with the fastText library using both *skipgram* and *CBOW*. We tested the following parameters:

- Five values of embeddings sizes: 25, 50, 100, 250, and 500;
- Four for the number of epochs: 5, 10, 20, and 50;
- Two for the learning rate: 0.05 and 0.1.

We considered subwords with 5 to 6 letters while setting the minimum word count as 100, running on an Intel Core i7 CPU equipped with 32 GB RAM. The best embeddings chosen by each measure for each dataset are reported in Table 3.

Comments on the Best Embedding To better clarify the matter, using the *ap* [69] dataset, in Fig.3 we provide a scatter plot produced over the best embedding model—as emerges from Table 3—generated with UMAP[2]. We choose twenty random records for the first ten categories. Each icon and colour is assigned to one category, demonstrating how well the clusters are separated from each other. Analysing the scatter plot, it can be observed that:

- Categories that are conceptually more distant with respect to the other categories show a clearer separation, for instance ♦ *chemical element* and × *monetary unit*, while categories semantically close together have less clear boundaries, such as ⬢ *legal documents* and ★ *assets*;
- There are cases that are on the borderline of two or more classes. While such cases may seem to show model weakness, such observations can be explained based on the nature of the non-contextual embeddings, since by definition they cannot represent words with multiple meanings. For example, in the scatter plot mentioned above, the word *capital*, which belongs to ★ *assets* in WordNet, is on the border of ♦ *social unit* and ⬢ *district*. This can be explained by the fact that we used the Wikipedia corpus (i.e. a generic corpus) for generating the embeddings.

Below, we describe the four downstream NLP tasks performed to extrinsically evaluate the embedding selection procedure. For each task, we comment on the results.

Categorisation

Following [46], we cluster the vectors from each selected embedding, applying the k-means algorithm from the

Table 3 Best embeddings for each measure/dataset

	type	size	epoch	learning rate
HSS (ours)	skipgram	500	5	0.05
MEN [62]	skipgram	250	10	0.05
SimLex-999 [48]	CBOW	500	50	0.01
WUP [31]	CBOW	50	10	0.01

[2] Uniform Manifold Approximation and Projection (UMAP) is a dimension reduction technique that can be used for visualisation similarly to t-SNE, but also for general nonlinear dimension reduction

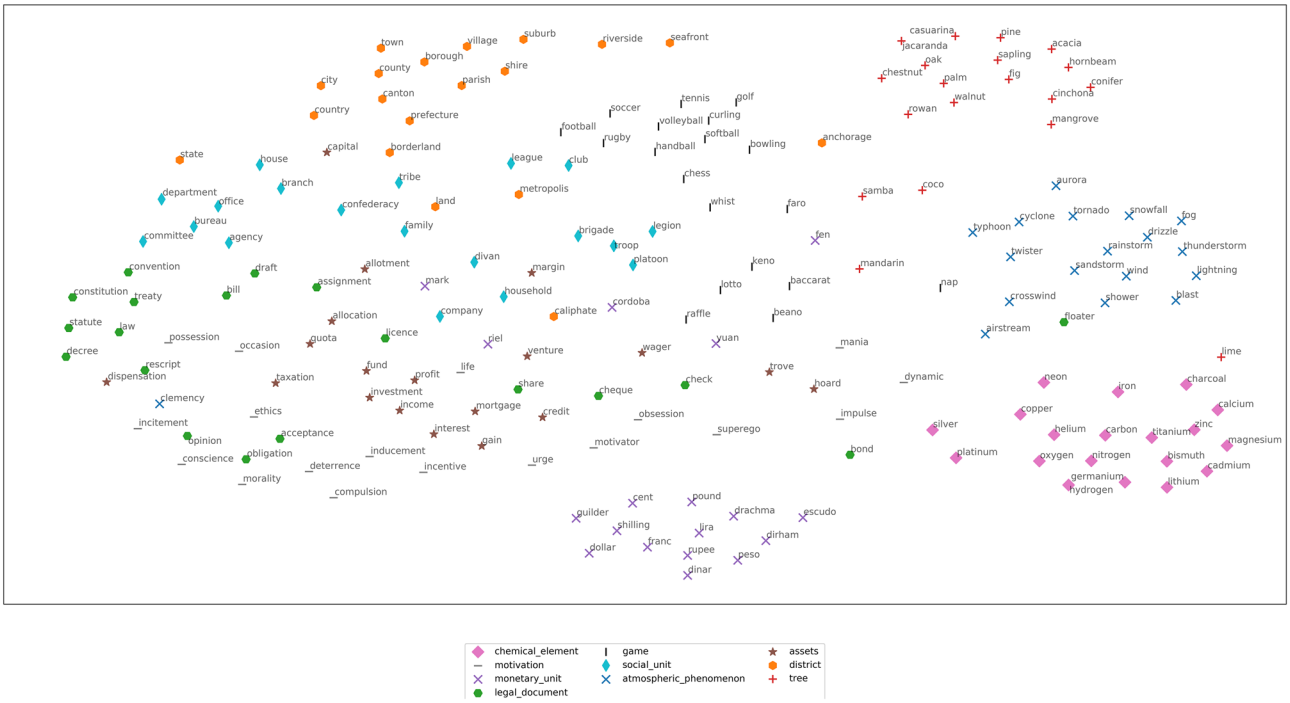

Fig. 3 UMAP plot of the **best** word-embedding model resulting from Table 3, that is *skipgram, dim=500, epochs=5 and learning rate = 0.05*. Each icon is assigned to one category in ap [69]

scikit-learn library [72], with the default parameters. In the next step, the purity of each cluster is calculated. The number of clusters is equal to the number of classes in each dataset. To account for the variation in results, we compute the average value of 50 iterations for each pair.

The datasets used in this experiments are **ap** [69], containing 403 concepts organised into 21 categories, **battig** [70], including 4,668 concepts belonging to 56 categories, and **esslli** [68], from the ESSLLI 2008 Distributional Semantic Workshop shared-task set, containing 45 concepts divided into 9 categories.

The purity values of clusters are reported in Table 4, where a purity close to 1 shows that the cluster is well reproduced, while a purity close to 0 indicates poor cluster quality. These results show that the embedding chosen by our similarity measure outperforms the other three embeddings, chosen by MEN, SimLex-999, and WUP measure when applied on three well-know categorisation datasets used by [46].

Table 4 Categorisation: Cluster purity obtained from each embedding/dataset

	Categorisation		
	ap [69]	battig [70]	**esslli** [68]
HSS (ours)	**0.75**	**0.62**	**0.81**
MEN [62]	0.71	0.58	0.77
SimLex-999 [48]	0.71	0.49	0.78
WUP [31]	0.68	0.43	0.73

Table 5 Sentiment Classification

	Accuracy	**Precision**	**Recall**	**F1**
	multiclass Amazon Review			
HSS (ours)	**0**.4 ± 0.04	**0**.4 ± 0.04	**0**.41 ± 0.04	**0**.4 ± 0.04
MEN [62]	0.4 ± 0.04	0.39 ± 0.04	0.4 ± 0.04	0.39 ± 0.04
Sim-Lex-999 [48]	**0**.41 ± 0.02	**0**.4 ± 0.02	**0**.41 ± 0.02	**0**.4 ± 0.02
WUP [31]	0.34 ± 0.06	0.33 ± 0.04	0.34 ± 0.06	0.31 ± 0.05
	Binary Amazon Review			
HSS (ours)	**0**.82 ± 0.02	**0**.81 ± 0.03	**0**.84 ± 0.03	**0**.82 ± 0.02
MEN [62]	0.81 ± 0.02	0.79 ± 0.03	0.83 ± 0.02	0.81 ± 0.02
SimLex-999 [48]	0.8 ± 0.04	0.79 ± 0.04	0.82 ± 0.05	0.8 ± 0.04
WUP [31]	0.72 ± 0.03	0.71 ± 0.04	0.74 ± 0.04	0.72 ± 0.03
	Binary Movie Review			
HSS (ours)	**0**.84 ± 0.01	**0**.84 ± 0.01	**0**.84 ± 0.02	**0**.84 ± 0.01
MEN [62]	0.82 ± 0.01	0.82 ± 0.01	0.82 ± 0.02	0.82 ± 0.01
SimLex-999 [48]	0.83 ± 0.01	0.83 ± 0.01	**0**.84 ± 0.02	0.83 ± 0.01
WUP [31]	0.72 ± 0.02	0.72 ± 0.02	0.72 ± 0.02	0.72 ± 0.02

Table 6 Hypernym detection

	Hyper/Hyponym Classification			
	Accuracy	Precision	Recall	F1
HSS (ours)	**0**.72 ± 0.013	**0**.72 ± 0.012	**0**.75 ± 0.017	**0**.73 ± 0.013
MEN [62]	0.71 ± 0.014	**0**.72 ± 0.017	0.72 ± 0.015	0.72 ± 0.012
SimLex-999 [48]	0.69 ± 0.023	0.68 ± 0.251	0.73 ± 0.017	0.70 ± 0.019
WUP [31]	0.68 ± 0.015	0.71 ± 0.02	0.65 ± 0.014	0.68 ± 0.013

Table 7 Synonym detection

	Synonym Classification			
	Accuracy	Precision	Recall	F1
HSS (ours)	**0**.616 ± 0.006	**0**.609 ± 0.007	**0**.624 ± 0.012	**0**.617 ± 0.006
MEN [62]	0.586 ± 0.006	0.58 ± 0.007	0.586 ± 0.008	0.583 ± 0.006
SimLex-999 [48]	0.612 ± 0.007	0.605 ± 0.008	0.618 ± 0.004	0.611 ± 0.005
WUP [31]	0.54 ± 0.007	0.534 ± 0.007	0.537 ± 0.014	0.536 ± 0.007

Sentiment Classification

We carry out the sentiment classification task on three user review datasets: Binary Movie Review, Binary, and Multiclass Amazon Review.

Replicating the experiment done in [6], we perform binary sentiment classification on the dataset from [73]. This dataset contains 50K movie reviews divided equally between binary labels. Utilising the embeddings as the features of a LIBLINEAR logistic regression [74], we compute a linear combination of embeddings, weighted by the word count in each review. We use the Scikit-learn library [72] to apply the mentioned regression and calculate the accuracy values for each selected embedding by performing a 10-fold cross-validation. The results of this task (classification accuracy) can be seen in Table 5.

Using the binary and the multiclass Amazon cellphone review datasets[3], we perform both binary and multiclass sentiment classification. These datasets contain 26, 845 and 29, 988 reviews labelled 0 or 1 for the binary dataset and from 1 (absolutely negative) to 5 (absolutely positive) for the multiclass dataset. In both cases, we downsample the dataset based on the minority label. Table 5 shows the mean and the standard deviation of the performance metrics that result from 10-fold cross-validation. Our measure can outperform MEN, SimLex-999, and WUP benchmarks. The exception is the multiclass dataset, in which HSS produces results similar to those of SimLex-999.

Hypernym Detection

For this task, we employ the BATS benchmark dataset [75], which contains 99,200 questions in 40 morphological and semantic categories. In particular, we use three lexicography categories, namely, *Hypernyms* (Animals, Miscellaneous) and *Hyponyms* (Miscellaneous).

To generate hypernym–hyponym pairs (corresponding to the positive pairs), we extract all the possible pairs, deduplicate them, and remove all the pairs with at least one word out of the embedding vocabulary. The process leads to 1,129 pairs. To generate the negative pairs, we use the aforementioned categories to randomly select words and form pairs, arriving at 1,129 pairs that do not have a hypernym–hyponym relationship. Finally, by subtracting the vectors of pair words, we create a single vector, and we use it as the feature for training the classifier. To compensate for the potential effect of the randomly generated pairs on the results, we repeat the process ten times, each time using a logistic regression model to classify the pairs.

Table 6 shows the mean and standard deviation of the outcomes for chosen embeddings. The HSS outperforms the other benchmarks except for the MEN dataset, which achieves the same precision as our method.

Synonym Detection

As for the previous task, we use the BATS benchmark to generate the training data for the logistic regression classifier. To create positive pairs (i.e. pairs with synonym relationship), first, we generate all the possible combinations of 2 for each entry in *Synonym-exact* and *Synonym-intensity* files. Then, we combine them with the pair made from the synonymous words in the *Antonym-gradable* file, which results in 4,663 pairs that we reduced to 4,011 pairs after deduplication. Similar to the method described in the previous task, we generate negative pairs by iterating through the vocabulary ten times.

[3] https://jmcauley.ucsd.edu/data/amazon/

As reported in Table 7, despite close results to the SimLex-999 dataset, the embedding chosen by the HSS carries out a better synonym classification than the other methods.

Conclusion and Future Work

In previous studies on embeddings evaluation, similarity relationships encoded in existing semantic hierarchies were not considered. We presented TaxoVec, an unsupervised method to generate and select embeddings that incorporates the similarity relationships encoded in an existing semantic hierarchy.

The contribution of our work goes towards three direction. First, we evaluated a measure of semantic similarity, the HSS, that we developed in [13]. Here, we show that this new measure outperforms the SOTA, showing a higher correlation with gold benchmarks of similarity between words. Second, we implemented the HSS and all the other benchmark measures of semantic similarity that we surveyed and employed as a fully deployed Python package, available at https://pypi.org/project/TaxoSS. The package is easy to install and ready to use. Its main functionalities and computational times are presented in "Step 2: A Tool for Computing Semantic Similarity" and Fig. 2. Finally, we successfully used the HSS as supervision to select the embedding that better preserves the taxonomic relations of similarity, i.e. we generated several embeddings, and we selected the one that better preserves the taxonomic similarity between words. We compared against previous literature over four downstream tasks: categorisation, sentiment classification, hypernym detection, and synonym detection. The embedding selected through TaxoVec outperforms the SOTA in all four of these benchmark NLP tasks.

We are currently working on three aspects. First, we are improving TaxoVec with the inclusion of hypernymy-hyponymy relations, second, we are working on a new version of TaxoVec that exploits context embeddings to represent multiple senses of taxonomic terms, and third, we are assessing the impact of TaxoVec on domain-specific applications, exploiting domain-specific taxonomies and corpora. Our plan is also to include those improvements the python package we created, TaxoSS.

Declarations

Ethical Approval This article does not contain any studies with human participants performed by any of the authors.

Conflict of Interest All authors declare that they have no conflict of interest.

References

1. Tang D, Qin B, Liu T. Document modeling with gated recurrent neural network for sentiment classification. In: EMNLP; 2015.
2. Gupta A, Zhang P, Lalwani G, Diab M. CASA-NLU: Context-Aware Self-Attentive Natural Language Understanding for Task-Oriented Chatbots. arXiv preprint arXiv:190908705. 2019.
3. Zhang Y, Gan Z, Fan K, Chen Z, Henao R, Shen D, et al. Adversarial feature matching for text generation. In: Proceedings of Conference on Machine Learning. JMLR. org; 2017.
4. Bakarov A. A survey of word embeddings evaluation methods. 2018. arXiv preprint http://arxiv.org/abs/180109536.
5. Perone CS, Silveira R, Paula TS. Evaluation of sentence embeddings in downstream and linguistic probing tasks. arXiv preprint arXiv:180606259. 2018.
6. Schnabel T, Labutov I, Mimno D, Joachims T. Evaluation methods for unsupervised word embeddings. In: EMNLP; 2015.
7. Levy O, Goldberg Y, Dagan I. Improving distributional similarity with lessons learned from word embeddings. TACL. 2015;3.
8. Caselles-Dupré H, Lesaint F, Royo-Letelier J. Word2vec applied to recommendation: Hyperparameters matter. In: RECSYS; 2018.
9. Zhang Y, Ahmed A, Josifovski V, Smola A. Taxonomy discovery for personalized recommendation. In: Proceedings of the 7th ACM international conference on Web search and data mining; 2014.
10. Hua W, Wang Z, Wang H, Zheng K, Zhou X. Understand short texts by harvesting and analyzing semantic knowledge. IEEE transactions on Knowledge and data Engineering. 2016;29(3).
11. Wu W, Li H, Wang H, Zhu KQ. Probase: A probabilistic taxonomy for text understanding. In: ACM SIGMOD; 2012.
12. Resnik P. Semantic similarity in a taxonomy: An information-based measure and its application to problems of ambiguity in natural language. JAIR. 1999;11.
13. Malandri L, Mercorio F, Mezzanzanica M, Nobani N. MEET: A Method for Embeddings Evaluation for Taxonomic Data. In: 2020 International Conference on Data Mining Workshops (ICDMW). IEEE; 2020. p. 31-8.
14. Giabelli A, Malandri L, Mercorio F, Mezzanzanica M, Seveso A. NEO: A Tool for Taxonomy Enrichment with New Emerging Occupations. In: International Semantic Web Conference. Springer; 2020. p. 568–84.
15. Giabelli A, Malandri L, Mercorio F, Mezzanzanica M, Seveso A. NEO: A System for Identifying New Emerging Occupation from Job Ads. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 35; 2021. p. 16035–7.
16. Seveso A, Mercorio F, Mezzanzanica M. A Human-AI Teaming Approach for Incremental Taxonomy Learning from Text. In: Zhou Z, editor. Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021. ijcai.org; 2021. p. 4917–8. Available from: https://doi.org/10.24963/ijcai.2021/690.
17. Malandri L, Mercorio F, Mezzanzanica M, Nobani N. TaxoRef: Embeddings Evaluation for AI-driven Taxonomy Refinement. InJoint European Conference on Machine Learning and Knowledge Discovery in Databases 2021 Sep 13 (pp. 612–627). Springer, Cham.
18. Giabelli A, Malandri L, Mercorio F, Mezzanzanica M, Seveso A. Skills2Graph: Processing million Job Ads to face the Job Skill Mismatch Problem. In: Zhou Z, editor. Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021. ijcai.org; 2021. p. 4984–7. Available from: https://doi.org/10.24963/ijcai.2021/708.
19. Giabelli A, Malandri L, Mercorio F, Mezzanzanica M, Seveso A. Skills2Job: A recommender system that encodes job

offer embeddings on graph databases. Appl Soft Comput. 2021;101:107049. Available from: https://doi.org/10.1016/j.asoc.2020.107049.
20. Otter DW, Medina JR, Kalita JK. A survey of the usages of deep learning for natural language processing. IEEE Transactions on Neural Networks and Learning Systems. 2020.
21. Malandri L, Xing FZ, Orsenigo C, Vercellis C, Cambria E. Public mood-driven asset allocation: The importance of financial sentiment in portfolio management. Cognitive Computation. 2018;10(6):1167–76.
22. Minaee S, Kalchbrenner N, Cambria E, Nikzad N, Chenaghlu M, Gao J. Deep Learning-based Text Classification: A Comprehensive Review. ACM Computing Surveys (CSUR). 2021;54(3):1–40.
23. Deng L, Liu Y. Deep learning in natural language processing. Springer; 2018.
24. Xing F, Malandri L, Zhang Y, Cambria E. Financial Sentiment Analysis: An Investigation into Common Mistakes and Silver Bullets. In: Proceedings of the 28th International Conference on Computational Linguistics; 2020. p. 978–87.
25. Cambria E, Li Y, Xing FZ, Poria S, Kwok K. SenticNet 6: Ensemble application of symbolic and subsymbolic AI for sentiment analysis. In: Proceedings of the 29th ACM international conference on information & knowledge management; 2020. p. 105-14.
26. Fu R, Guo J, Qin B, Che W, Wang H, Liu T. Learning semantic hierarchies via word embeddings. In: ACL; 2014.
27. Maedche A, Volz R. The ontology extraction & maintenance framework Text-To-Onto. In: Proc. Workshop on Integrating Data Mining and Knowledge Management, USA; 2001.
28. Lastra-Díaz JJ, García-Serrano A, Batet M, Fernández M, Chirigati F. HESML: A scalable ontology-based semantic similarity measures library with a set of reproducible experiments and a replication dataset. Information Systems. 2017;66.
29. Aouicha MB, Taieb MAH, Hamadou AB. SISR: System for integrating semantic relatedness and similarity measures. Soft Computing. 2018;22(6).
30. Leacock C, Chodorow M. Combining local context and WordNet similarity for word sense identification. WordNet: An electronic lexical database. 1998;49(2).
31. Wu Z, Palmer M. Verbs semantics and lexical selection. In: ACL; 1994.
32. Jiang JJ, Conrath DW. Semantic similarity based on corpus statistics and lexical taxonomy. arXiv preprint cmp-lg/9709008. 1997.
33. Lin D, et al. An information-theoretic definition of similarity. In: ICML. vol. 98; 1998.
34. Seco N, Veale T, Hayes J. An intrinsic information content metric for semantic similarity in WordNet. In: Ecai. vol. 16; 2004.
35. Collobert R, Weston J. A unified architecture for natural language processing: Deep neural networks with multitask learning. In: ICML. ACM; 2008.
36. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J. Distributed representations of words and phrases and their compositionality. In: NeurIPS; 2013.
37. Pennington J, Socher R, Manning C. Glove: Global vectors for word representation. In: EMNLP; 2014.
38. Levy O, Goldberg Y. Neural word embedding as implicit matrix factorization. In: NeurIPS; 2014.
39. Bojanowski P, Grave E, Joulin A, Mikolov T. Enriching word vectors with subword information. ACL. 2017;5.
40. Faruqui M, Dodge J, Jauhar SK, Dyer C, Hovy E, Smith NA. Retrofitting word vectors to semantic lexicons. arXiv preprint arXiv:14114166. 2014.
41. Kiela D, Hill F, Clark S. Specializing word embeddings for similarity or relatedness. In: EMNLP; 2015.
42. Nguyen KA, Köper M, Walde SSi, Vu NT. Hierarchical embeddings for hypernymy detection and directionality. arXiv preprint arXiv:170707273. 2017.
43. Meng Y, Huang J, Wang G, Zhang C, Zhuang H, Kaplan L, et al. Spherical text embedding. In: Advances in Neural Information Processing Systems; 2019. p. 8208-17.
44. Devlin J, Chang MW, Lee K, Toutanova K. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:181004805. 2018.
45. Wang B, Wang A, Chen F, Wang Y, Kuo CCJ. Evaluating word embedding models: methods and experimental results. APSIPA Transactions on Signal and Information Processing. 2019;8.
46. Baroni M, Dinu G, Kruszewski G. Don't count, predict! a systematic comparison of context-counting vs. context-predicting semantic vectors. In: ACL; 2014.
47. Agirre E, Alfonseca E, Hall K, Kravalova J, Paşca M, Soroa A. A study on similarity and relatedness using distributional and WordNet-based approaches. In: NAACL; 2009. p. 19–27.
48. Hill F, Reichart R, Korhonen A. Simlex-999: Evaluating semantic models with (genuine) similarity estimation. Computational Linguistics. 2015;41(4).
49. Liza FF, Grzes M. An improved crowdsourcing based evaluation technique for word embedding methods. In: Workshop on Evaluating Vector-Space Representations for NLP; 2016.
50. Köhn A. What's in an embedding? Analyzing word embeddings through multilingual evaluation. In: EMNLP; 2015.
51. Lau JH, Baldwin T. An empirical evaluation of doc2vec with practical insights into document embedding generation. arXiv preprint arXiv:160705368. 2016.
52. Press O, Wolf L. Using the output embedding to improve language models. arXiv preprint arXiv:160805859. 2016.
53. Ghannay S, Favre B, Esteve Y, Camelin N. Word embedding evaluation and combination. In: LREC; 2016.
54. AlMousa M, Benlamri R, Khoury R. Exploiting non-taxonomic relations for measuring semantic similarity and relatedness in WordNet. Knowledge-Based Systems. 2021;212:106565.
55. Schönbrodt FD, Perugini M. At what sample size do correlations stabilize? Journal of Research in Personality. 2013;47(5).
56. Camacho-Collados J, Pilehvar MT, Collier N, Navigli R. Semeval-2017 task 2: Multilingual and cross-lingual semantic word similarity. In: Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017); 2017. p. 15-26.
57. Xia Y, Cambria E, Hussain A, Zhao H. Word polarity disambiguation using bayesian model and opinion-level features. Cognitive Computation. 2015;7(3).
58. Valdivia A, Luzón MV, Cambria E, Herrera F. Consensus vote models for detecting and filtering neutrality in sentiment analysis. Information Fusion. 2018;44:126–35.
59. Wang Z, Ho SB, Cambria E. Multi-level fine-scaled sentiment sensing with ambivalence handling. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems. 2020;28(04):683–97.
60. Miller GA, Charles WG. Contextual correlates of semantic similarity. Language and cognitive processes. 1991;6(1).
61. Bird S, Klein E, Loper E. Natural language processing with Python: analyzing text with the natural language toolkit. O'Reilly Media, Inc. 2009.
62. Bruni E, Tran NK, Baroni M. Multimodal distributional semantics. JAIR. 2014;49.
63. Rubenstein H, Goodenough JB. Contextual correlates of synonymy. Communications of the ACM. 1965;8(10).
64. Finkelstein L, Gabrilovich E, Matias Y, Rivlin E, Solan Z, Wolfman G, et al. Placing search in context: The concept revisited. In: WWW; 2001.

65. Radinsky K, Agichtein E, Gabrilovich E, Markovitch S. A word at a time: computing word relatedness using temporal semantic analysis. In: WWW; 2011.
66. Halawi G, Dror G, Gabrilovich E, Koren Y. Large-scale learning of word relatedness with constraints. In: ACM SIGKDD; 2012.
67. Cohen J. A power primer. Psychological bulletin. 1992;112(1).
68. Baroni M, Evert S, Lenci A. Bridging the gap between semantic theory and computational simulations: Proceedings of the esslli workshop on distributional lexical semantics. Hamburg, Germany: FOLLI. 2008.
69. Almuhareb A. Attributes in lexical acquisition. University of Essex; 2006.
70. Baroni M, Murphy B, Barbu E, Poesio M. Strudel: A distributional semantic model based on property and types. Cognitive Science. 2010;34(2).
71. Aranganayagi S, Thangavel K. Clustering Categorical Data Using Silhouette Coefficient as a Relocating Measure. In: International Conference on Computational Intelligence and Multimedia Applications (ICCIMA 2007). vol. 2; 2007. p. 13–7.
72. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: Machine learning in Python. the Journal of machine Learning research. 2011;12.
73. Maas AL, Daly RE, Pham PT, Huang D, Ng AY, Potts C. Learning word vectors for sentiment analysis. In: ACL HLT; 2011.
74. Fan RE, Chang KW, Hsieh CJ, Wang XR, Lin CJ. LIBLINEAR: A library for large linear classification. Journal of machine learning research. 2008;9(Aug).
75. Gladkova A, Drozd A, Matsuoka S. Analogy-based detection of morphological and semantic relations with word embeddings: what works and what doesn't. In: NAACL; 2016.